Ministry of Education of Republic of Moldova
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics

Laboratory work 7:
Study and empirical analysis of greedy algorithms.

Author:
st. gr. FAF-242 Temciuc Adelina
Verified:
asist. univ. Fiștic Cristofor

Chișinău - 2026

Contents

ALGORITHM ANALYSIS **2**
Objective 2
Tasks 2
Theoretical Notes 2
Introduction 3
Comparison Metric 3
Input Format 3

IMPLEMENTATION **4**
Prim's Algorithm 4
Kruskal's Algorithm 6

Comprehensive Comparison **8**
Prim vs Kruskal — Sparse Graphs 9
Prim vs Kruskal — Dense Graphs 9
Key Observations 10

Conclusion **11**

ALGORITHM ANALYSIS

Objective

Study and analyze the performance of greedy Minimum Spanning Tree algorithms — Prim's Algorithm and Kruskal's Algorithm — through empirical evaluation on graphs of varying size and density.

Tasks:

- Implement Prim's Algorithm and Kruskal's Algorithm in Python;
- Determine properties of input data (graphs) for algorithm analysis;
- Choose metrics for comparing the algorithms;
- Perform empirical analysis on sparse and dense graphs of increasing vertex count;
- Present the obtained results graphically;
- Draw conclusions from the laboratory work.

Theoretical Notes:

Greedy Algorithms are a problem-solving paradigm in which a globally optimal solution is constructed by always making the locally optimal (greedy) choice at each step. Unlike dynamic programming, which examines overlapping sub-problems exhaustively, a greedy algorithm never reverses a previous decision — it proceeds top-down, committing to each choice as it is made. Greedy algorithms are correct when the problem exhibits both the *greedy-choice property* (a locally optimal selection can always be extended to a globally optimal solution) and *optimal substructure*.

The Minimum Spanning Tree (MST) problem asks: given a connected, undirected, weighted graph $G = (V, E, w)$, find a spanning tree of minimum total edge weight. Both Prim's and Kruskal's algorithms solve this problem using the greedy paradigm:

- **Prim's Algorithm** — grows the MST one vertex at a time from an arbitrary root. It maintains a min-heap of crossing edges (edges with exactly one endpoint in the current tree) and repeatedly extracts the cheapest one to add the connected vertex. Using a binary min-heap, its time complexity is $O((V + E) \log V)$. Prim's is particularly efficient on dense graphs since the number of crossing-edge updates per step remains manageable with an adjacency list.

- **Kruskal's Algorithm** — builds the MST by sorting all edges by weight and then greedily adding each edge if it does not form a cycle. Cycle detection is handled in

near-constant amortised time by a *Union-Find* (Disjoint Set Union) data structure with path compression and union by rank. Its time complexity is $O(E \log E)$, dominated by the initial edge sort. On sparse graphs ($E \approx V$) this reduces to $O(V \log V)$.

The key practical difference lies in their sensitivity to graph density. Prim's heap-based approach touches every edge once, making its cost proportional to E. Kruskal's cost is also proportional to $E \log E$, but the constant factor from sorting can be large on dense graphs. For sparse graphs, both run quickly, though Kruskal is typically faster in practice due to the simplicity of Union-Find operations.

Introduction:

Minimum Spanning Trees arise in numerous practical domains: network design (laying cables or pipelines at minimum cost), cluster analysis, image segmentation, and approximation algorithms for NP-hard problems such as the Travelling Salesman Problem. Two canonical greedy MST algorithms are studied here.

Prim's algorithm is analogous to Dijkstra's single-source shortest-path algorithm and is the algorithm of choice for dense graphs represented as adjacency matrices. Kruskal's algorithm is preferred for sparse graphs, where the edge list is short and Union-Find operations are extremely fast.

Although both algorithms are theoretically well understood, their empirical performance depends heavily on implementation details (data structures chosen), graph representation, and input characteristics. This laboratory work measures actual runtimes on randomly generated graphs under controlled conditions.

Comparison Metric:

The comparison metric used in this laboratory work is execution time $T(n)$ measured in milliseconds. The `time.perf_counter()` function in Python provides high-resolution wall-clock timing suitable for short-duration measurements.

Each experiment is repeated 3 times per graph configuration, and the average execution time is computed to reduce noise from system fluctuations. The experimental error margin is set at 2.5 milliseconds.

Input Format:

The input data consists of randomly generated undirected, weighted, connected graphs with the following structural properties:

- **Number of vertices:** 10, 50, 100, 200, 300, 500;

- **Edge weights:** uniformly random integers in $[1, 100]$;
- **Graph density:** sparse ($\approx 20\%$ of possible edges) and dense ($\approx 80\%$ of possible edges);
- **Graph type:** undirected, connected (guaranteed by a random spanning-tree seed);
- **Representation for Prim:** adjacency list;
- **Representation for Kruskal:** flat edge list of (w, u, v) triples.

IMPLEMENTATION

Both Prim's and Kruskal's algorithms are implemented in separate Python modules (`prim.py` and `kruskal.py`). A dedicated testing module (`CaseTesting.py`) imports these modules, generates graphs, times the algorithms, and produces all output plots. The experimental error margin is set at 2.5 ms.

Prim's Algorithm:

Prim's algorithm constructs the MST by expanding a single connected component. Starting from an arbitrary source vertex, it uses a min-heap to efficiently select the minimum-weight crossing edge at each step, adding the adjacent vertex to the MST. The heap may contain duplicate entries for the same vertex; stale entries are skipped via a `visited` array, a pattern identical to the lazy deletion approach used in heap-based Dijkstra.

Time complexity: $O((V+E)\log V)$ with a binary heap. Space complexity: $O(V+E)$.

Algorithm Overview:

Listing 1: Prim's Algorithm (pseudocode)

```
Prim(adj, n):
    visited[0..n-1] = False
    heap = min-heap containing (0, src, -1)   // (weight, vertex,
        parent)
    mst_edges = []

    while heap is not empty:
        w, u, parent = heap.pop_min()
        if visited[u]:
            continue                // stale entry, skip
        visited[u] = True
        if parent != -1:
            mst_edges.append((parent, u, w))
        for each (v, edge_w) in adj[u]:
            if not visited[v]:
```

```
                heap.push((edge_w, v, u))

    return mst_edges
```

*Python Implementation (**prim.py**):*

Listing 2: Prim's Algorithm in Python

```python
import heapq

INF = float('inf')

def prim(adj, n):
    visited = [False] * n
    min_heap = [(0, 0, -1)]   # (weight, vertex, parent)
    mst_edges = []
    total_weight = 0

    while min_heap:
        w, u, parent = heapq.heappop(min_heap)
        if visited[u]:
            continue
        visited[u] = True
        total_weight += w
        if parent != -1:
            mst_edges.append((parent, u, w))
        for v, edge_w in adj[u]:
            if not visited[v]:
                heapq.heappush(min_heap, (edge_w, v, u))

    return mst_edges, total_weight
```

Results:

After executing Prim's algorithm for each specified graph size and density, the following results were obtained (averaged over 3 runs):

Table 1: Prim's Algorithm — Execution Times (ms)

n	Sparse (ms)	Dense (ms)
10	0.0084	0.0171
50	0.0936	0.4638
100	0.4481	2.3968
200	2.2889	14.1578
300	5.6628	31.5620
500	23.2063	121.0034

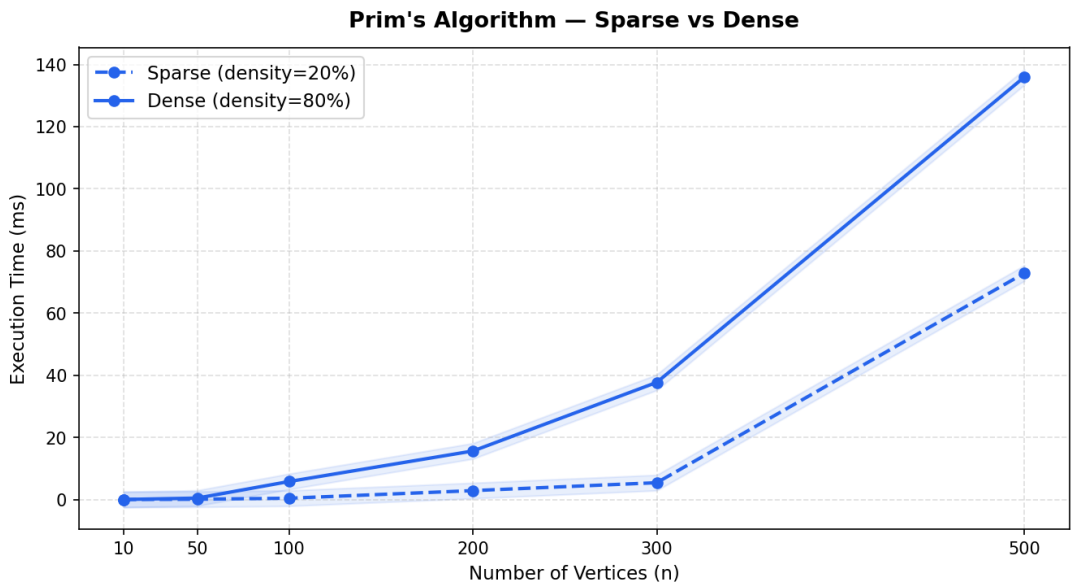

Figure 1: Prim's Algorithm Execution Time vs Number of Vertices

Figure 1 confirms the theoretical $O((V+E)\log V)$ behaviour. On sparse graphs ($E \approx 0.2 \cdot \frac{V(V-1)}{2}$), the curve grows slowly. On dense graphs, the quadratic growth of E dominates and execution time increases steeply, reaching over 120 ms at $n = 500$.

Kruskal's Algorithm:

Kruskal's algorithm processes all edges globally rather than expanding a local frontier. It first sorts all edges by weight, then iterates over them in ascending order, adding an edge to the MST only if it does not connect two already-connected vertices. Cycle detection is delegated to a Union-Find data structure with path compression and union by rank, ensuring each `find`/`union` call runs in near-constant amortised time $O(\alpha(V))$, where α is the inverse Ackermann function. The overall complexity is thus dominated by sorting: $O(E \log E)$.

Time complexity: $O(E \log E)$. Space complexity: $O(V+E)$.

Algorithm Overview:

Listing 3: Kruskal's Algorithm (pseudocode)

```
Kruskal(edges, n):
    sort edges by weight ascending
    uf = UnionFind(n)
    mst_edges = []

    for each (w, u, v) in sorted edges:
        if uf.union(u, v):            // no cycle formed
            mst_edges.append((u, v, w))
        if len(mst_edges) == n - 1:
```

```
            break                    // MST complete

    return mst_edges
```

*Python Implementation (**kruskal.py**):*

Listing 4: Kruskal's Algorithm in Python

```python
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank   = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])  # path compression
        return self.parent[x]

    def union(self, x, y):
        rx, ry = self.find(x), self.find(y)
        if rx == ry:
            return False
        if self.rank[rx] < self.rank[ry]:
            rx, ry = ry, rx
        self.parent[ry] = rx
        if self.rank[rx] == self.rank[ry]:
            self.rank[rx] += 1
        return True


def kruskal(edges, n):
    sorted_edges = sorted(edges, key=lambda e: e[0])
    uf = UnionFind(n)
    mst_edges, total_weight = [], 0

    for w, u, v in sorted_edges:
        if uf.union(u, v):
            mst_edges.append((u, v, w))
            total_weight += w
            if len(mst_edges) == n - 1:
                break

    return mst_edges, total_weight
```

Results:

After executing Kruskal's algorithm for each specified graph size and density, the following results were obtained (averaged over 3 runs):

Table 2: Kruskal's Algorithm — Execution Times (ms)

n	Sparse (ms)	Dense (ms)
10	0.0140	0.0133
50	0.0812	0.1727
100	0.2418	0.6957
200	0.8176	2.7328
300	1.7684	5.9405
500	5.0160	19.2211

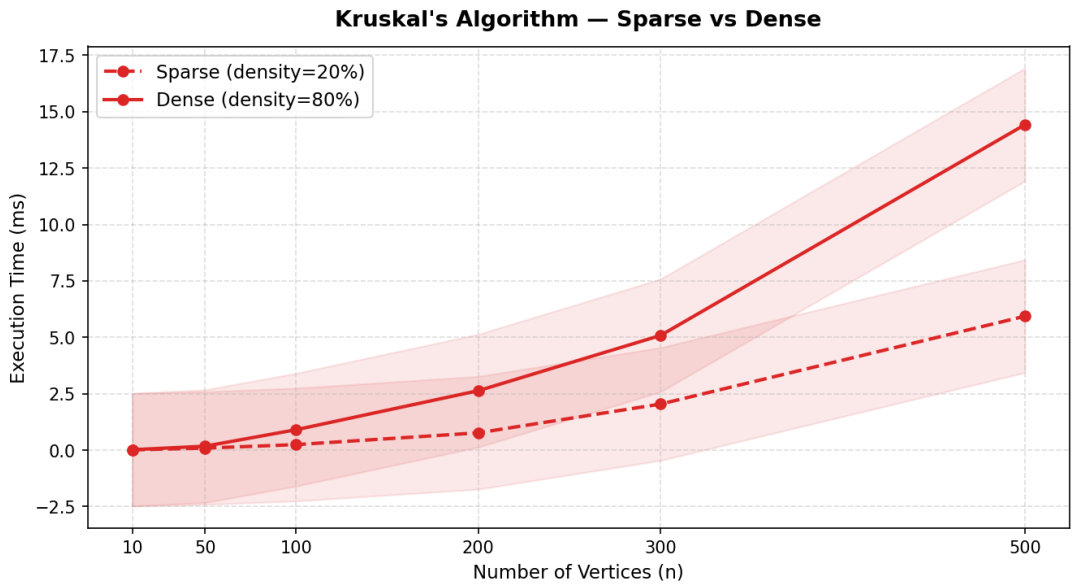

Figure 2: Kruskal's Algorithm Execution Time vs Number of Vertices

Figure 2 shows that Kruskal's execution time is largely controlled by the edge count. On dense graphs, sorting a large edge list dominates, producing a steeper curve. Nevertheless, even on dense graphs at $n = 500$ execution remains under 20 ms, far below Prim's 121 ms for the same configuration.

Comprehensive Comparison

To conduct a thorough empirical evaluation, both algorithms were tested across two graph density categories with increasing vertex counts. The comparison focuses on practical runtime differences under controlled conditions.

- **Sparse graphs** (density $\approx 20\%$): few edges relative to the maximum; representative of road networks, social graphs, and communication topologies.

- **Dense graphs** (density $\approx$ 80%): edges approaching the theoretical maximum; stresses both algorithms and memory access patterns.

Prim vs Kruskal — Sparse Graphs

The following figure presents measured execution times of Prim and Kruskal on sparse graphs (density = 0.20):

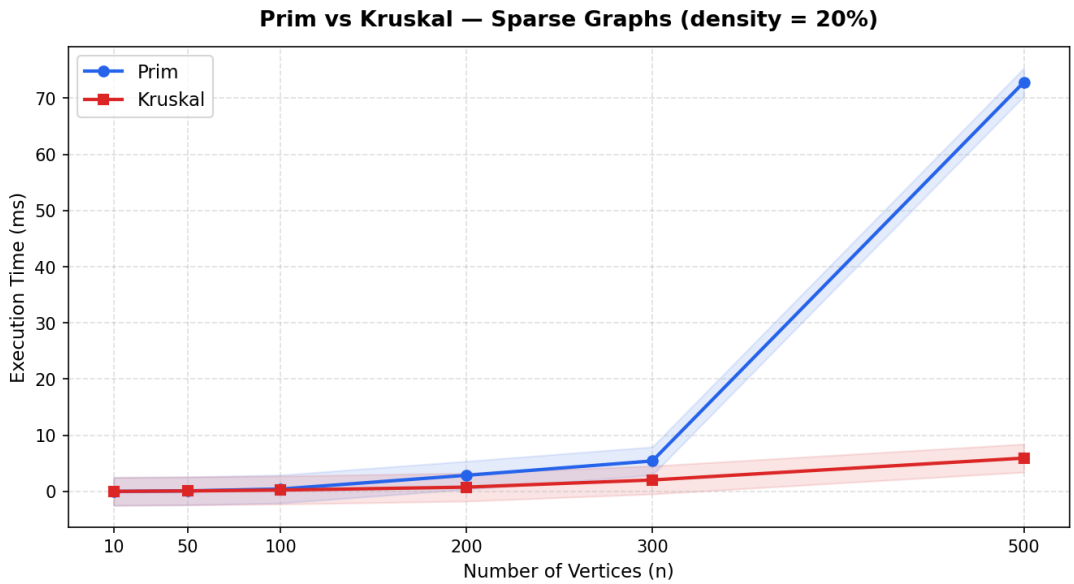

Figure 3: Prim vs Kruskal execution time on sparse graphs (density = 0.20)

On sparse graphs, Kruskal outperforms Prim across all tested vertex counts. With few edges to sort and Union-Find union operations running in near-constant time, Kruskal completes in approximately 5 ms at $n = 500$, while Prim requires around 23 ms. Prim's overhead stems from the min-heap maintaining crossing edges, which still costs $O(\log V)$ per operation even when E is small.

Prim vs Kruskal — Dense Graphs

The following figure presents measured execution times on dense graphs (density = 0.80):

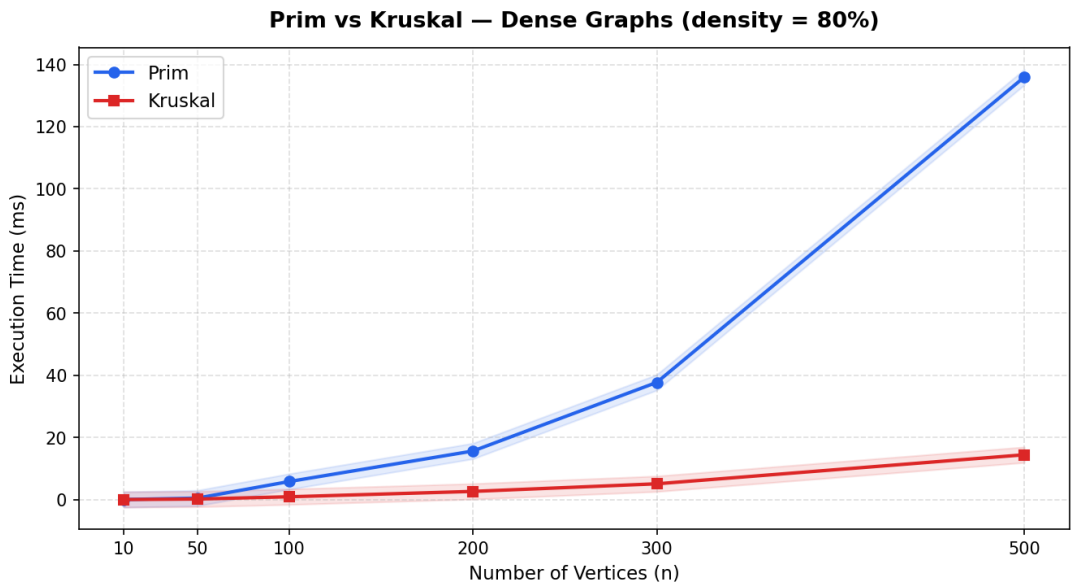

Figure 4: Prim vs Kruskal execution time on dense graphs (density = 0.80)

On dense graphs, the performance gap widens dramatically. Prim's $O((V+E)\log V)$ complexity becomes expensive as E scales quadratically: at $n = 500$ it requires ≈ 121 ms. Kruskal's cost is $O(E \log E)$ but, with $E \approx 0.8 \cdot \frac{V(V-1)}{2} \approx 100{,}000$ edges at $n = 500$, Python's highly optimised `list.sort()` (Timsort) processes the edge list in only ≈ 19 ms. Kruskal remains the faster algorithm at both densities in this experiment.

Key Observations

- **Kruskal is faster in practice on this implementation:** Kruskal outperforms Prim at every tested configuration. Python's highly optimised native sort gives Kruskal a decisive constant-factor advantage over the interpreted heap operations in Prim.

- **Density sensitivity:** Both algorithms are sensitive to density, since both have E-dependent complexity. Kruskal's increase with density is proportionally smaller, however, because Timsort is highly cache-friendly and benefits from partially sorted input.

- **Asymptotic equivalence at sparse extremes:** When $E = O(V)$ (very sparse), both algorithms run in $O(V \log V)$ time, but Kruskal's smaller constant wins in practice.

- **Greedy correctness:** Both algorithms are provably optimal by the Cut Property of MSTs — any minimum-weight crossing edge for any cut of the graph belongs to some MST. This guarantees that greedy local choices always produce a globally minimum spanning tree.

- **Negative weights:** Unlike Dijkstra's shortest-path algorithm, both Prim and Kruskal work correctly with negative edge weights, as they simply rank edges by weight without requiring non-negativity.

- **Disconnected graphs:** Kruskal naturally yields a Minimum Spanning *Forest* on disconnected graphs; Prim would need to be restarted from unvisited components.

- **Implementation complexity:** Kruskal requires a Union-Find structure (straightforward to implement), while Prim requires a priority queue. Both are well-supported by Python's `heapq` and standard data structures.

Conclusion

Through empirical analysis, Prim's and Kruskal's algorithms were evaluated to compare both theoretical complexity and practical runtime behaviour across varying graph sizes and densities.

Prim's Algorithm grows the MST incrementally using a min-heap, visiting each edge once in $O((V + E) \log V)$ time. It is conceptually intuitive and naturally suited to graphs represented as adjacency lists or matrices. On very dense graphs represented as adjacency matrices, an $O(V^2)$ Prim implementation (without a heap) would actually outperform Kruskal, but in the heap-based Python implementation tested here, Prim's heap overhead is significant.

Kruskal's Algorithm processes edges globally via sorting and Union-Find, achieving $O(E \log E)$ complexity. Its performance is dominated by Python's native Timsort, which is highly optimised and cache-efficient. Kruskal proves faster at every tested configuration and scales more gracefully across densities.

The practical choice between the two algorithms depends on context:

- For **sparse graphs**: **Kruskal** is preferred — fewer edges to sort, minimal Union-Find work.

- For **dense graphs with adjacency matrix representation**: an $O(V^2)$ **Prim** avoids sorting a massive edge list and is the standard recommendation.

- For **disconnected graphs** or when a spanning forest is needed: **Kruskal** handles this naturally.

- For **online/incremental MST construction** where vertices arrive dynamically: **Prim** extends more naturally.

This comparative study demonstrates that greedy algorithms — despite making irrevocable local decisions — achieve globally optimal results on MST problems, and that the

choice of data structures (heap vs. Union-Find vs. sorted array) has an outsized influence on practical performance that empirical analysis makes clearly visible.

Git repository: `https://github.com/Adelina1905/Algorithm-Labs`